



ОНЛАЙН-ОБРАЗОВАНИЕ



Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐+ если все хорошо
Ставьте ☐- если есть проблемы

Не забыть включить запись!





Транзакции. Spring Tx

Иванов Владимир

Цели занятия

- Основа транзакций в реляционных БД
- Spring TX на основе Spring JDBC
- Уровни изоляций транзакций

Планы

- Теория на самом деле не жесть, жесть начинается на практике
- Много теории, Spring немного
- Но это очень важно. Spring Boot тут уже не спасет
- Домашки нет 🥳

Совсем чуть-чуть оргвопросов

- Срок проверки 3 дня, стараемся успевать. Домашки группа делает хорошо, у преподавателей много работы
- Текущие вопросы в чат, более общие в telegram. У нас уже комьюнити
- Со всех отзывов.

Поехали



Основа транзакций



Упражнение для разминки

Что по Вашему такое транзакция?

ACID

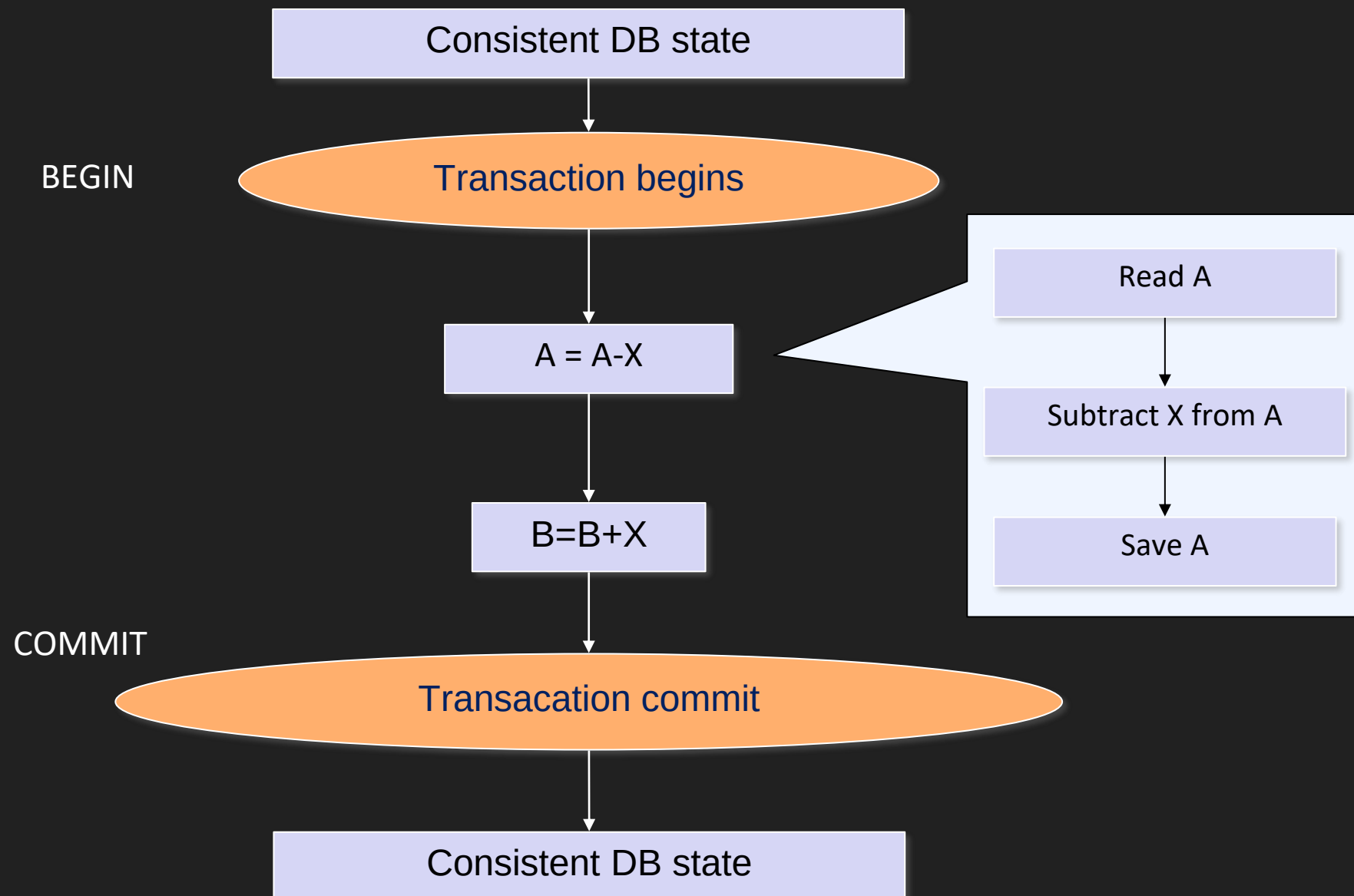
- Atomicity - атомарность
- Consistency - консистентность
- Isolation - изоляция
- Durability - долговечность

ACID

- ACID – очень важные
- Часто в БД «транзакции» – это не транзакции.
- В NoSQL базах данных – это совсем маркетинговая уловка
- Как ни странно это всё работает не только в БД

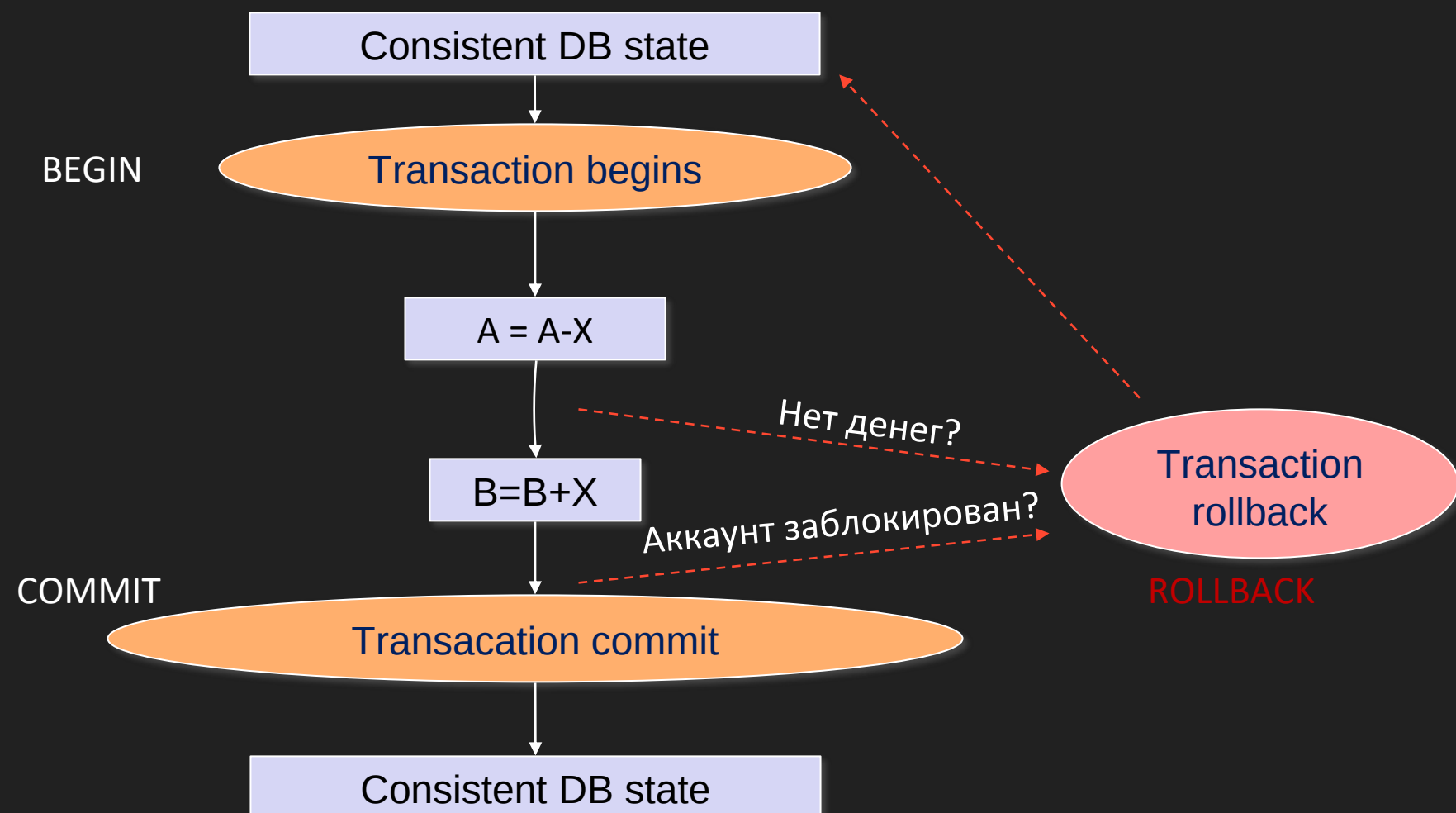
Commit

Перевод X долларов с аккаунта A на аккаунт B



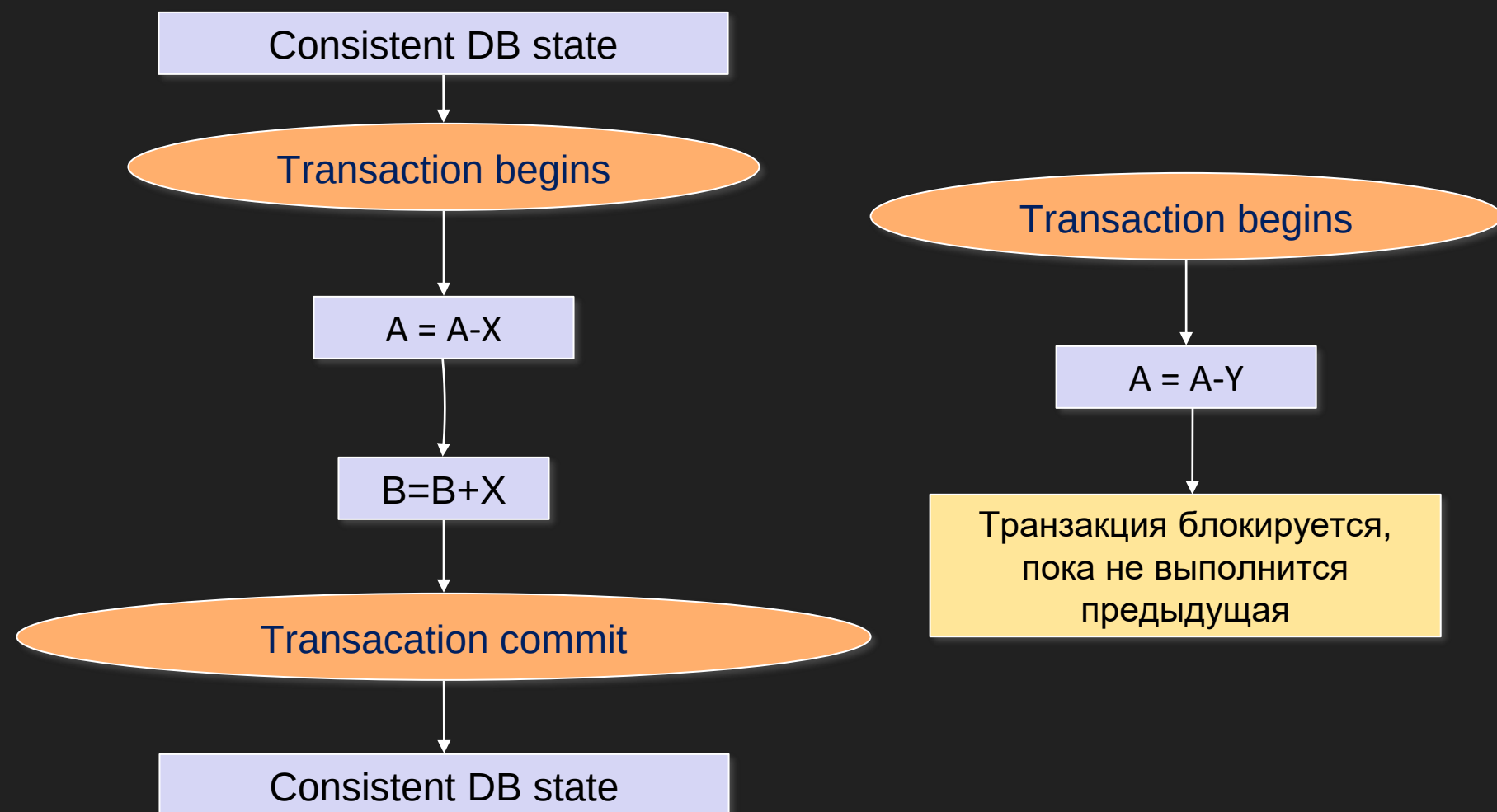
Rollback

Перевод X долларов с аккаунта A на аккаунт B



Параллельные транзакции

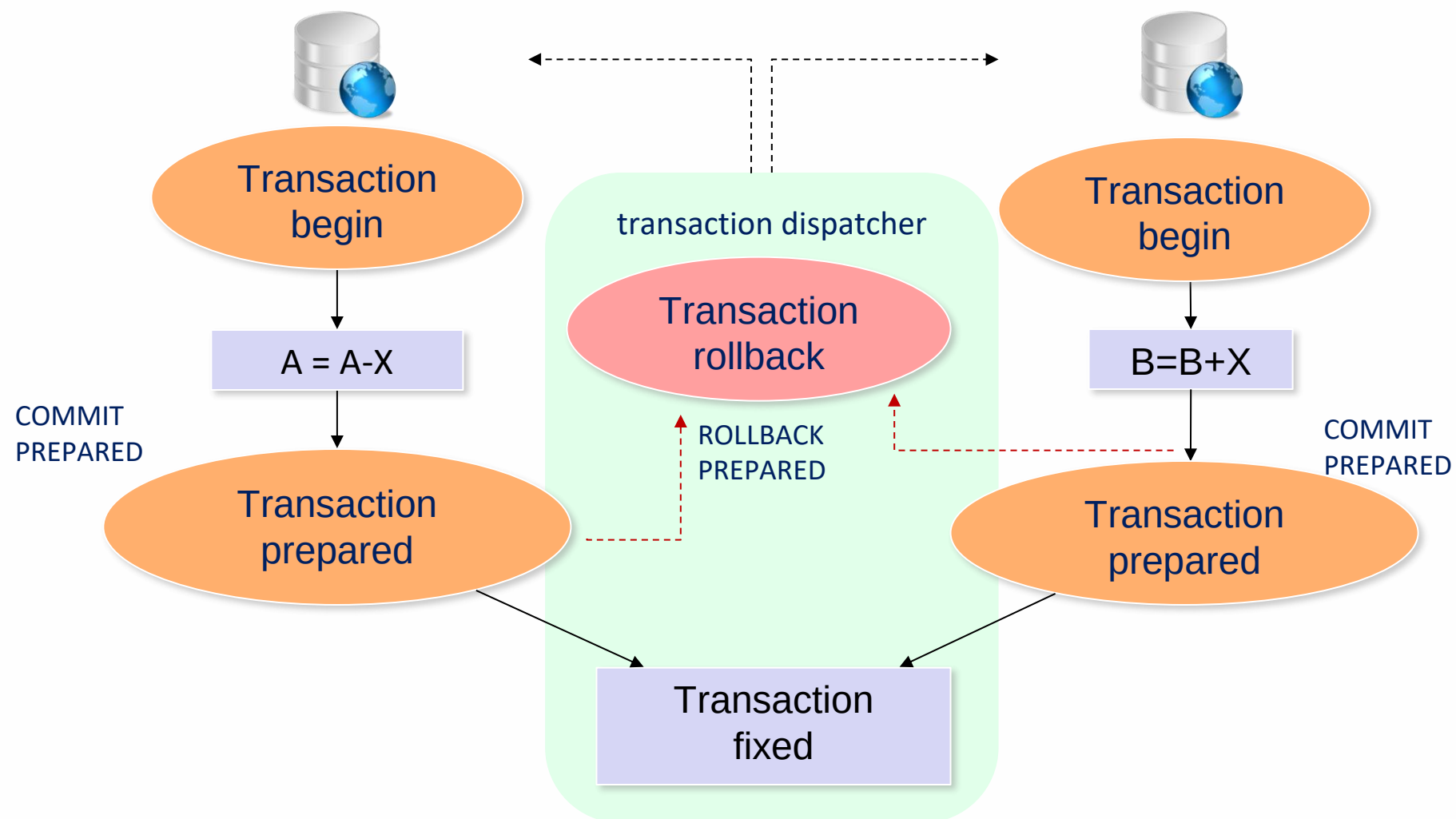
Перевод X долларов с аккаунта A на аккаунт B



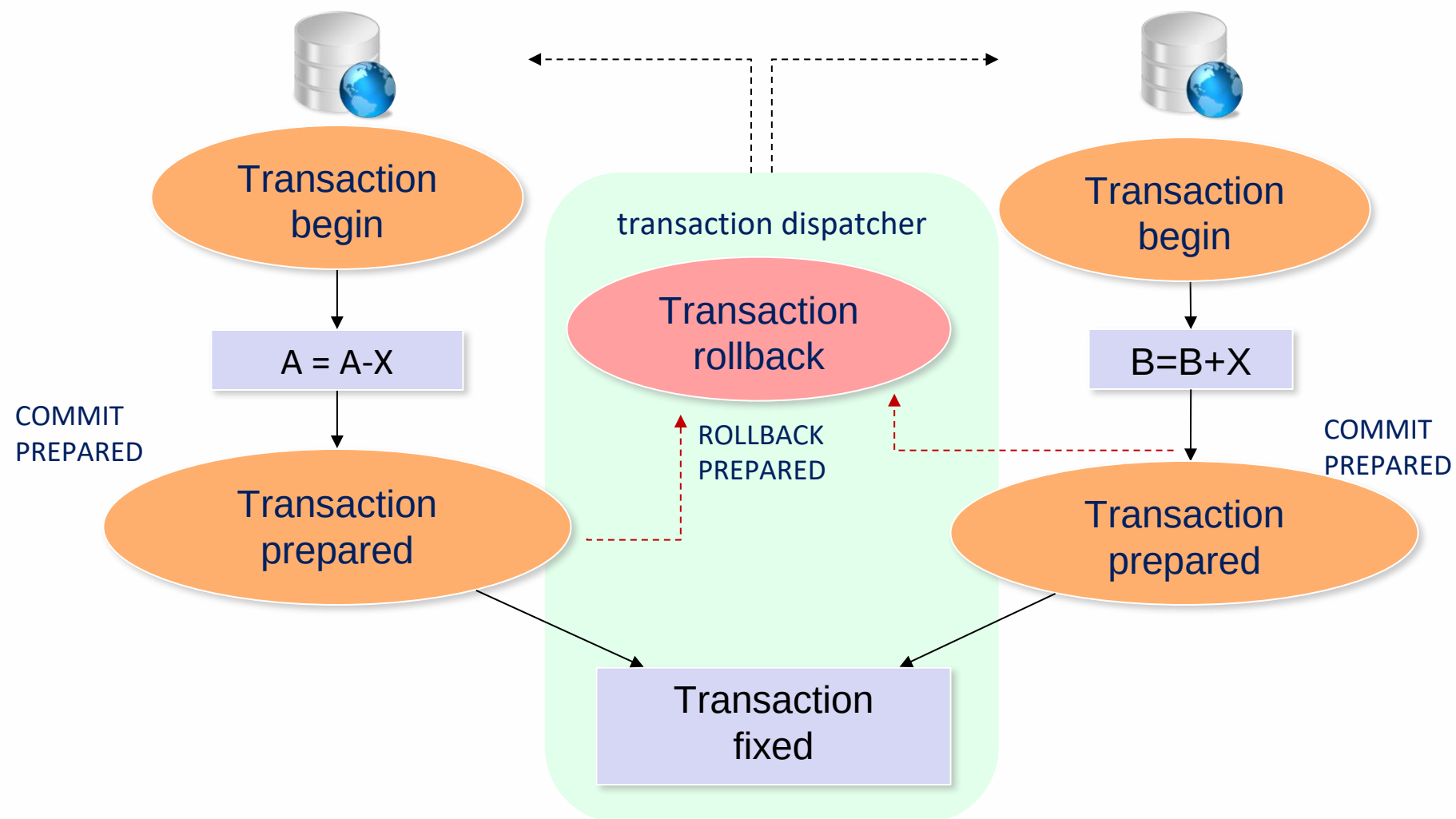
Типы транзакции

- Local – транзакции в одном DataSource
- Global – глобальные (распределённые), в нескольких DataSource-ах.
- Обычно управляются J2EE контейнером, используя JTA (Java Transaction API).
- Spring Boot позволяет просто подключать глобальные транзакции
- Но это совсем не просто...

Перевод X долларов с аккаунта A в одном банке на B в другом банке



Перевод X долларов с аккаунта A в одном банке на B в другом банке



- В действительности транзакции бывают не только у БД
- Есть транзакции и JMS
- Как думаете для чего?

- Транзакционность очень тяжело тестируется
- Дефекты иногда невозможно воспроизвести*
- Баг с транзакциями может искаться полгода*
- Unit-тестами почти не покрывается
- Но это ооочень важно в реальных приложениях

Ваши вопросы?

Если что – их можно задать потом

Spring Tx

- Spring Tx – модуль для управления транзакциями (spring-tx, часть spring-jdbc)
- Поддерживает транзакции для разных технологий – JDBC, JPA и т.д.
- TransactionManager управляет транзакциями

- DataSourceTransactionManager
- HibernateTransactionManager
- JmsTransactionManager
- JmsTransactionManager102
- JpaTransactionManager

- Декларативное управление с помощью аннотации `@Transactional`
(Часто незаслуженно не любят за «магию»)
- Есть возможность и императивного управления – `TransactionTemplate`
(Вполне объективно не любят за бОльшее количество кода)

```
@Transactional(readonly = true)
public class DefaultFooService implements FooService {

    ... public Person getPerson(String name) {
        ... // do something
    }

    ... @Transactional(readonly = false, propagation = Propagation.REQUIRES_NEW)
    ... public void update(Person person) {
        ... // do something
    }
}
```

@Transactional

- `org.springframework.transaction.annotation.Transactional`
- Не путать с `javax.transaction.Transactional`
- Хотя поведение схоже
- Опишите как она реализуется в Spring? Мы уже знаем.

@Transactional

Параметры аннотации @Transactional:

- isolation – уровень изоляции транзакции
- propagation – как вкладываются транзакции
- timeout
- readOnly
- И другие

@Transactional

@Transactional применяется к:

- Интерфейсам
 - Классам
 - Методам интерфейса
 - public методам класса (!)
-
- Считается хорошим тоном применять @Transactional на конкретные методы, вместо классов и интерфейсов

- Чтобы это заработало нужен тот самый TransactionManager
- Для разных технологий – он разный
- spring-boot-starter-tx включает TransactionManager для каждой транзакции

Ваши вопросы?

Если что – их можно задать потом

Изоляция транзакций



- Транзакции очень дорогие для БД.
 - Чтобы выполнить их корректно, по-хорошему их нужно выполнять последовательно
 - Никакая БД не выполняет последовательно, иначе её никто не купит.
 - Приходится идти на маркетинговые уловки
- *Тот самый @Transactional(isolation=)

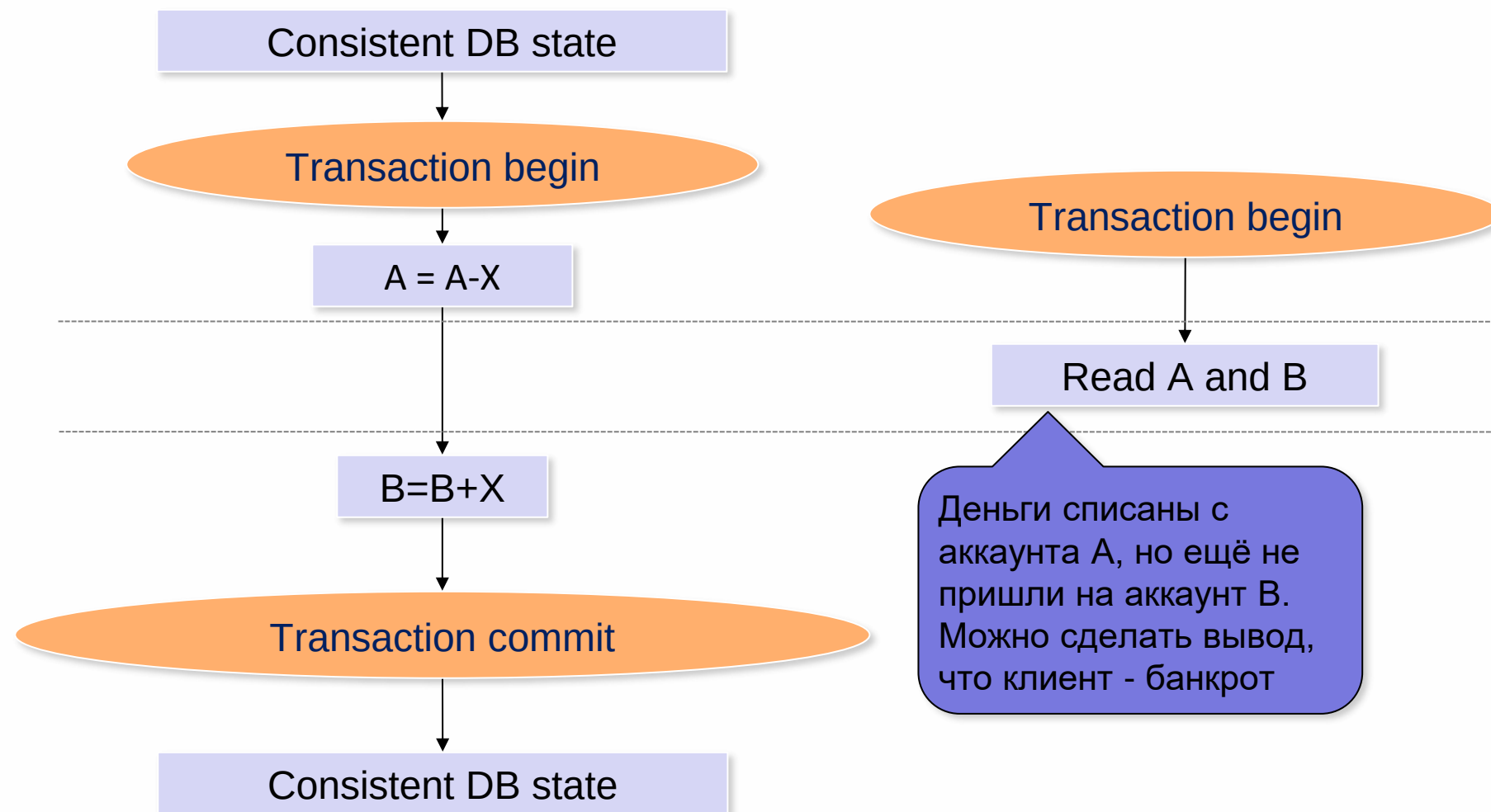
- Уровни изоляции транзакций – это как раз та уловка
- В действительности они не изолированы
- Но появляются какие-то side-эффекты, с которыми можно мириться в некоторых запросах
- За них БД получает плюшки, а от них – скорость.
- БД работает на своём уровне (любимый вопрос на собеседовании – на каком?)

«+» — предотвращает, «-» — не предотвращает.

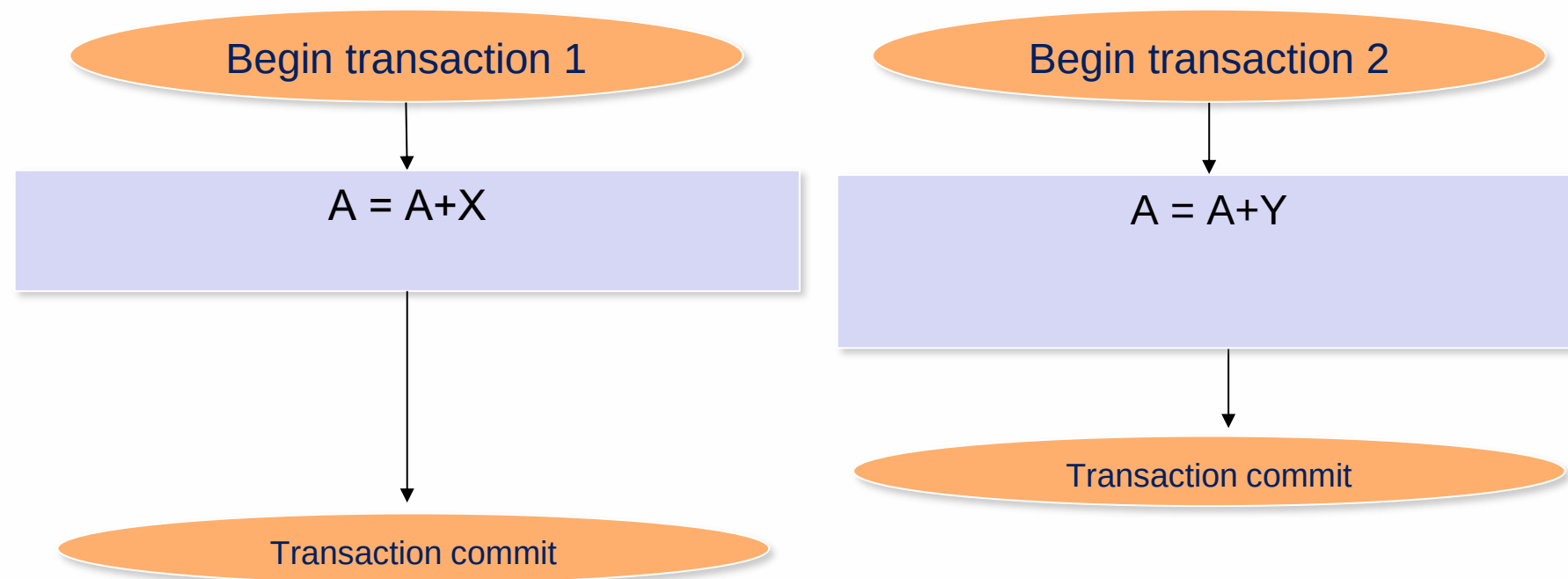
Уровень изоляции	Фантомное чтение	Неповторяющееся чтение	Потерянное обновление	«Грязное чтение»
READ UNCOMMITTED	-	-	-	-
READ COMMITTED	-	-	-	+
REPEATABLE READ	-	+	+	+
SERIALIZABLE	+	+	+	+

Dirty read (грязное чтение)

Перевод X долларов с аккаунта A на B

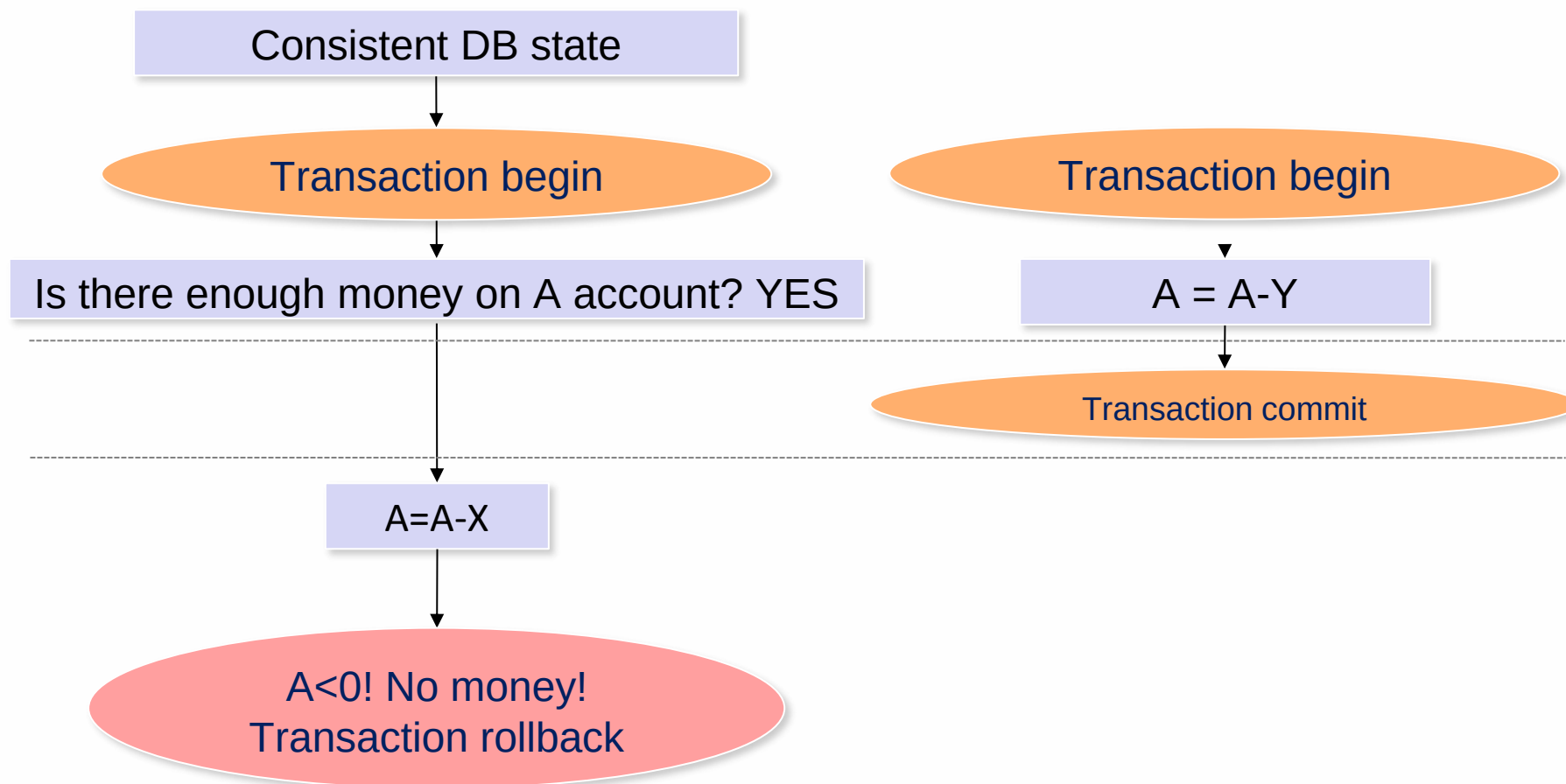


Какой будет результат?



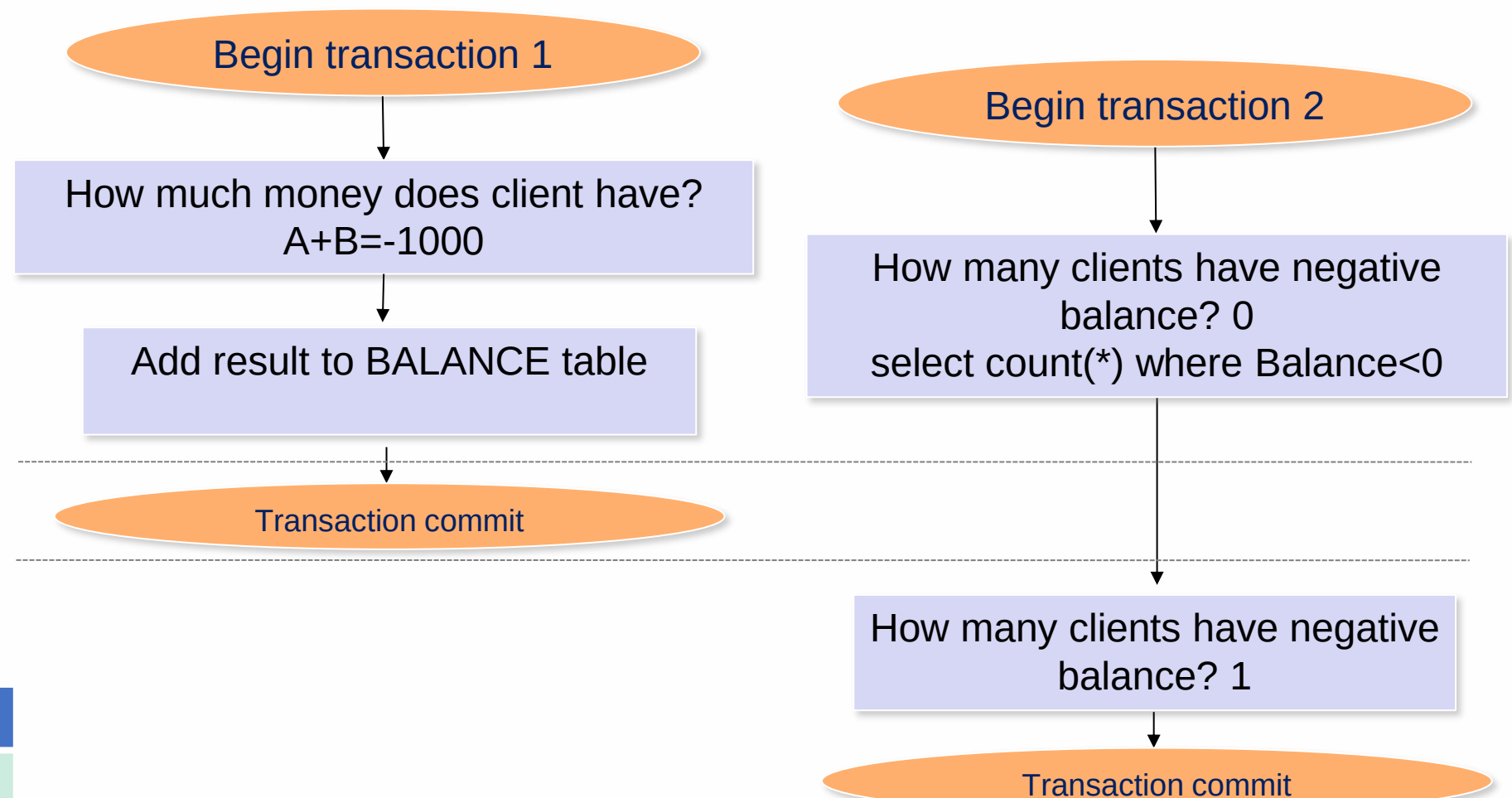
Unrepeatable read (неповторяемое чтение)

Снятие денег с аккаунта A двумя параллельными транзакциями:



Phantom read (фантомное чтение)

Во время второй транзакции появляются фантомные записи



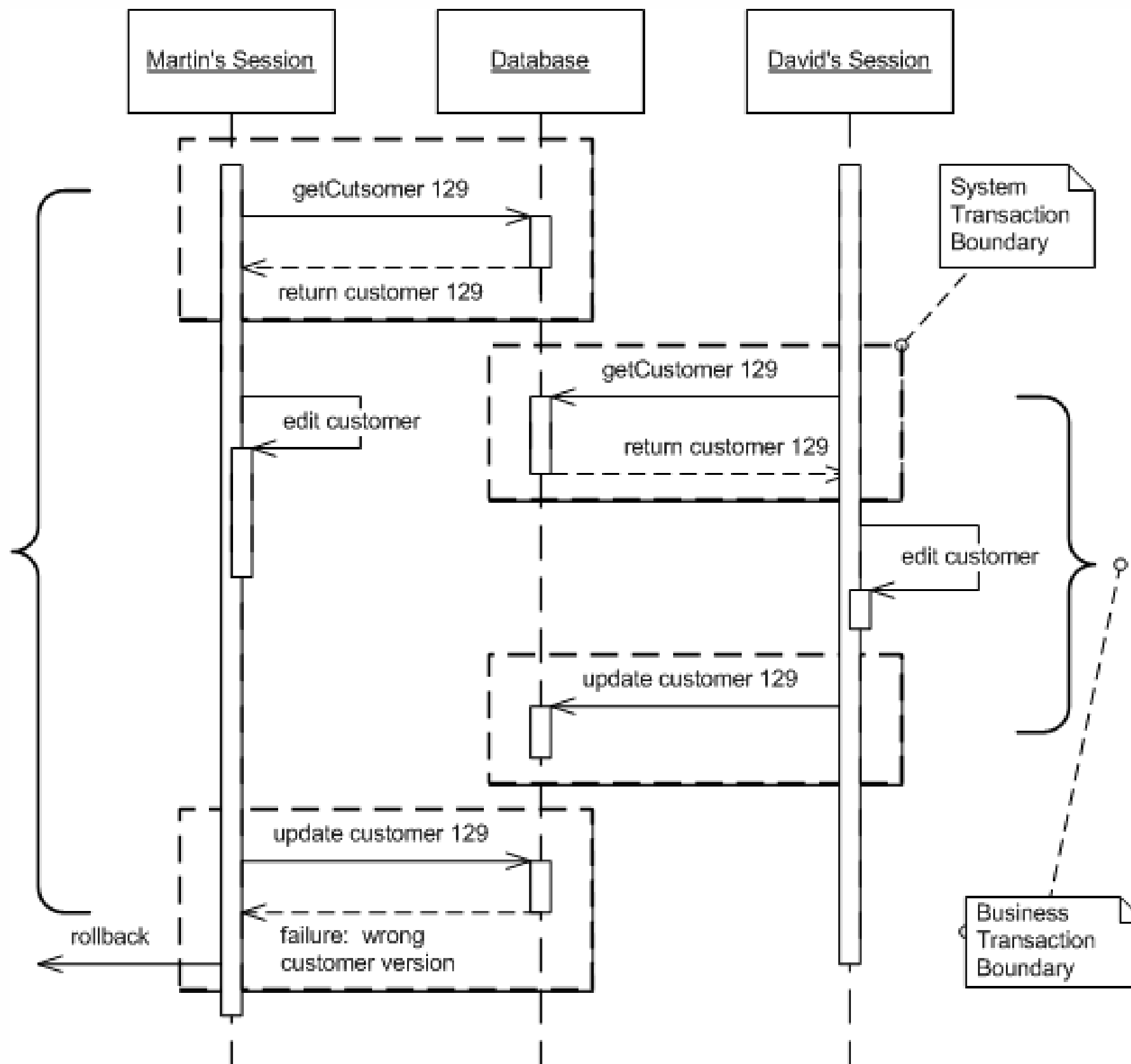
Surname	Balance
Smith	500
Johnson	-1000
Baker	9000

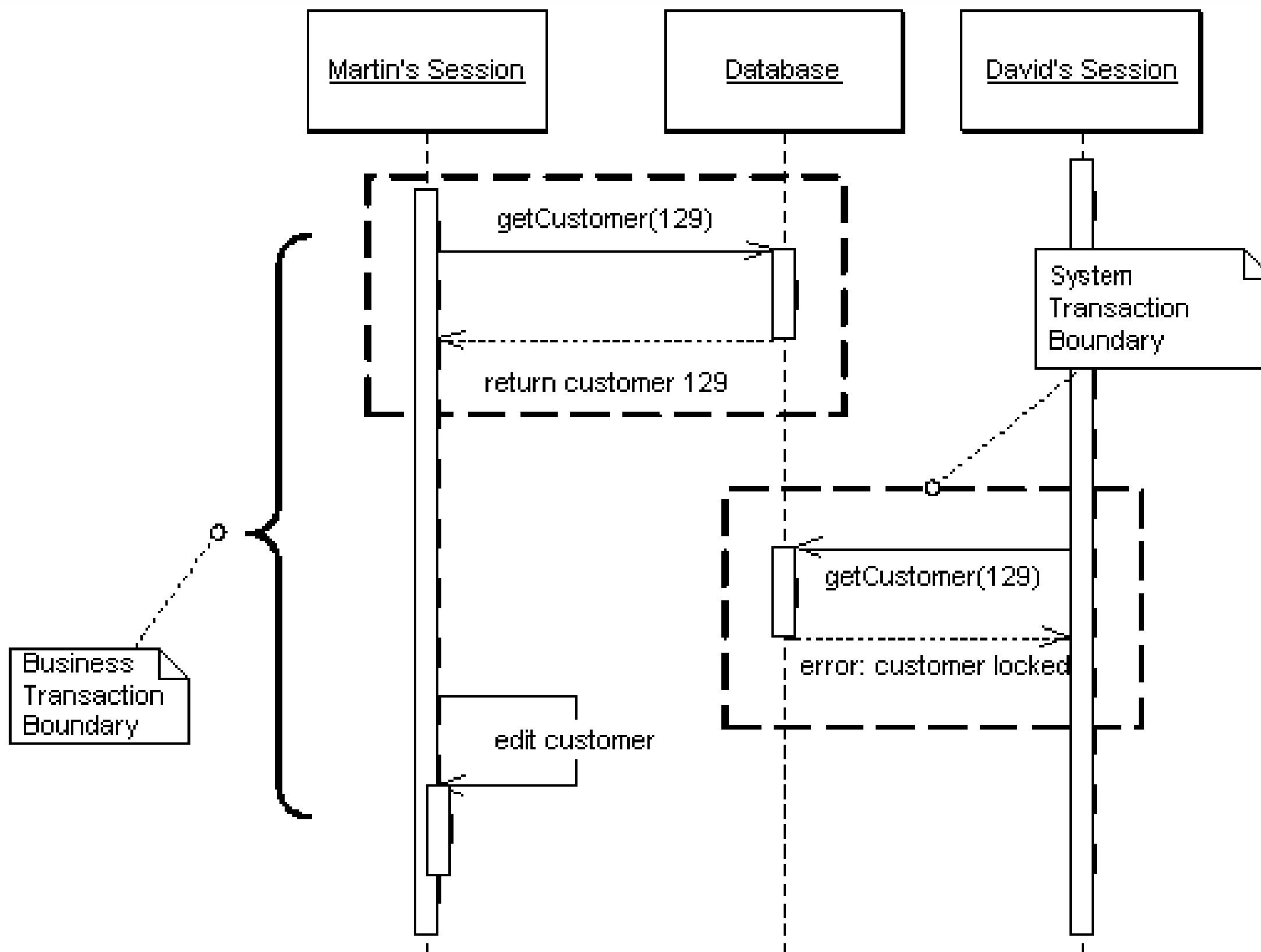
«+» — предотвращает, «-» — не предотвращает.

Уровень изоляции	Фантомное чтение	Неповторяющееся чтение	Потерянное обновление	«Грязное чтение»
READ UNCOMMITTED	-	-	-	-
READ COMMITTED	-	-	-	+
REPEATABLE READ	-	+	+	+
SERIALIZABLE	+	+	+	+

Ваши вопросы?

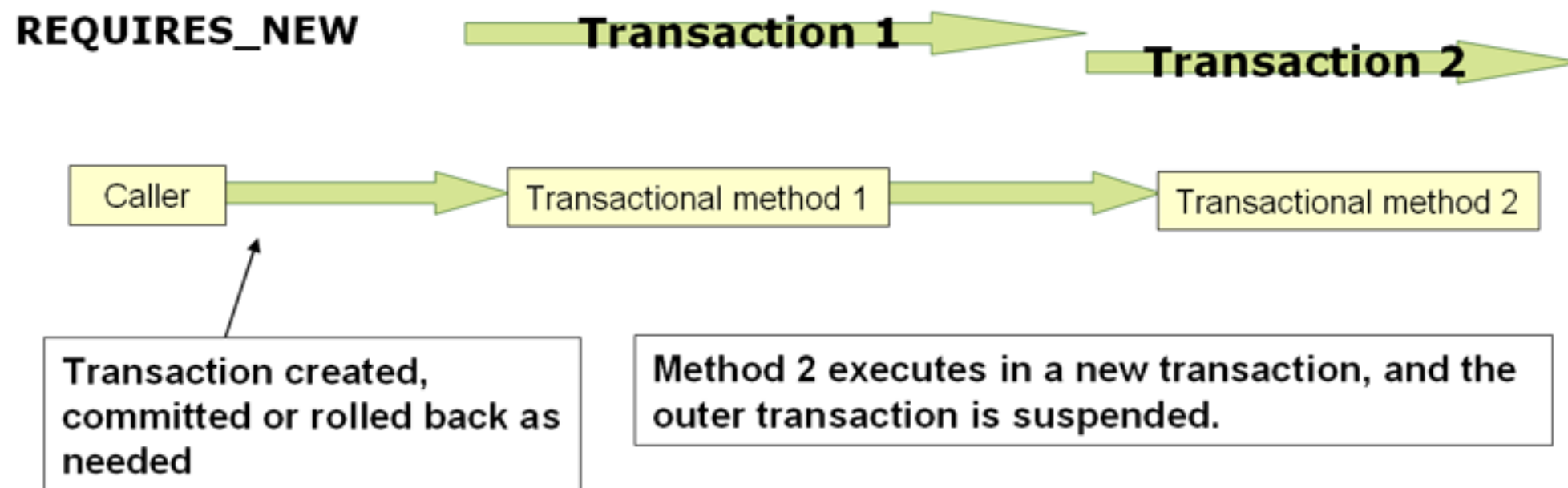
Если что – их можно задать потом

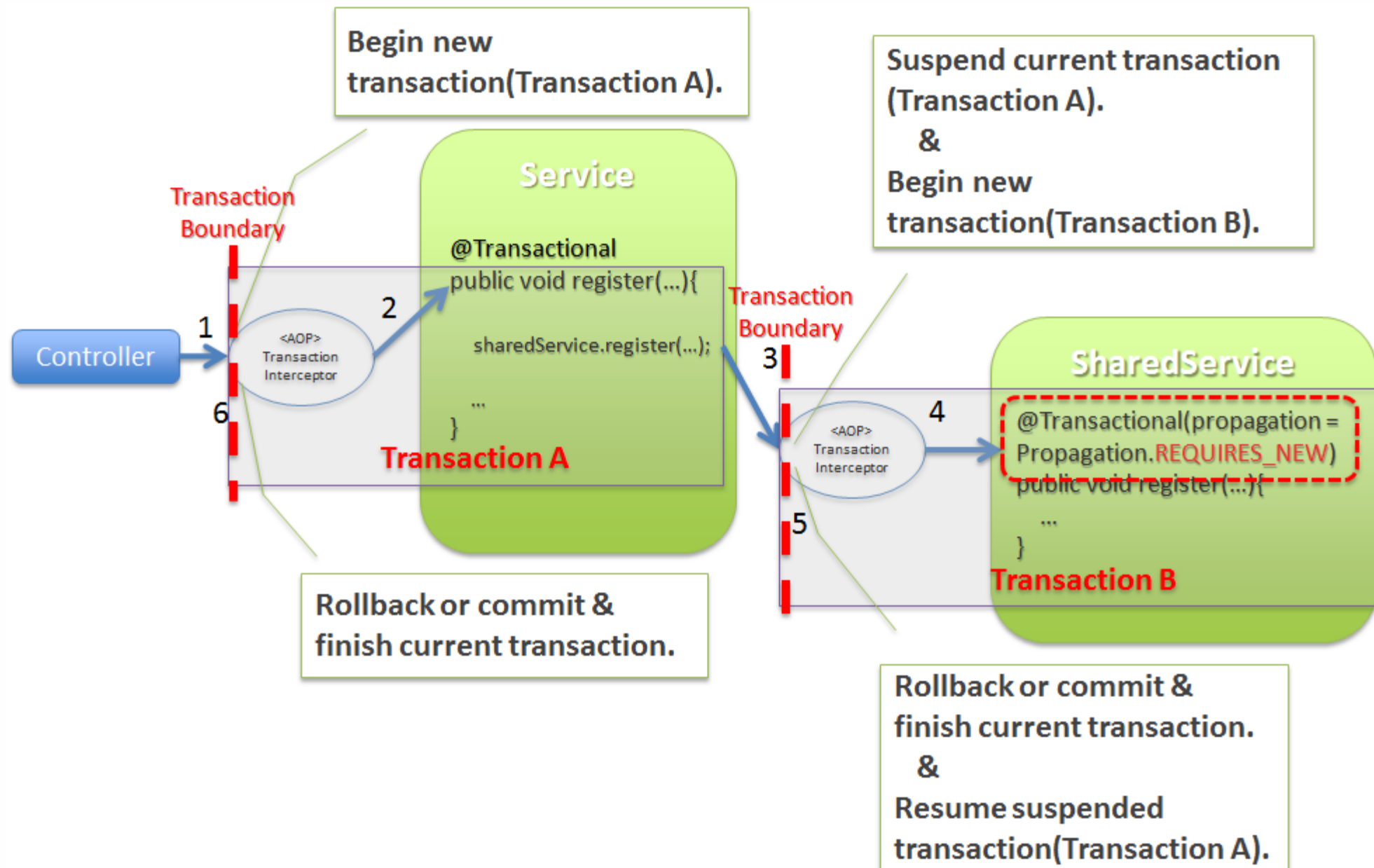




Ваши вопросы?

Если что – их можно задать потом





- Аспекты Spring применяются только при вызове из другого бина
- Вызов `private`, да другого `public` не приведёт к созданию транзакций
- Cglib может, но Spring не использует его
ВОЗМОЖНОСТЬ.

- Required
- RequiresNew
- Mandatory
- NotSupported
- Supports
- Never
- Nested

Propagation в @Transactional	Вызывающий метод	Вызываемый метод
Required – если транзакция начата – использует её. Если нет – начинает новую	No	T2
	T1	T1
RequiresNew – если метод вызван в транзакции – приостанавливает её и начинает новую, как только метод заканчивается, то исходная транзакция возобновляется.	No	T2
	T1	T2
Mandatory – если нет транзакции, то бросается exception.	No	Error
	T1	T1
NotSupported – приостанавливает текущую транзакцию, если была начата.	No	No
	T1	No
Supports – использует текущую транзакцию, если не была начата, то не открывает новую.	No	No
	T1	T1
Never – если запускается в транзакции, то бросается исключение.	No	No
	T1	Error
Nested – запускается во внутренней транзакции, если текущая существует,	No	T2
	T1	T2 (внутр.)

Propagation в @Transactional	Вызывающий метод	Вызываемый метод
Required – если транзакция начата – использует её. Если нет – начинает новую	No	T2
	T1	T1
RequiresNew – если метод вызван в транзакции – приостанавливает её и начинает новую, как только метод заканчивается, то исходная транзакция возобновляется.	No	T2
	T1	T2
Mandatory – если нет транзакции, то бросается exception.	No	Error
	T1	T1
NotSupported – приостанавливает текущую транзакцию, если была начата.	No	No
	T1	No
Supports – использует текущую транзакцию, если не была начата, то не открывает новую.	No	No
	T1	T1
Never – если запускается в транзакции, то бросается исключение.	No	No
	T1	Error
Nested – запускается во внутренней транзакции, если текущая существует,	No	T2
	T1	T2 (внутр.)

Ваши вопросы?

Если что – их можно задать потом

Rollback транзакций

```
@Service
public class OrderServiceImpl implements OrderService {

    @Autowired
    private Dao<OrderItem> dao;

    @Override
    @Transactional(rollbackFor = InvalidOrderItemException.class)
    public void persistOrders(List<OrderItem> orderItems) throws InvalidOrderItemException {
        for (OrderItem orderItem : orderItems) {
            if (orderItem.getQty() > 100) {
                throw new InvalidOrderItemException(
                    "Order quantity cannot be more than 100, found: "
                    + orderItem.getQty());
            }
            long id = dao.save(orderItem);
            System.out.println("id generated: " + id);
        }
    }
}
```

Rollback транзакций

- Как ни странно при checked-исключениях транзакции не откатываются!
- При checked- происходит commit!
- А вот при runtime – откатываются)
- Но это поведение можно перегрузить.
- Есть rollbackFor и noRollbackFor

Rollback транзакций

```
@Service
public class OrderServiceImpl implements OrderService {

    @Autowired
    private Dao<OrderItem> dao;

    @Override
    @Transactional(rollbackFor = InvalidOrderItemException.class)
    public void persistOrders(List<OrderItem> orderItems) throws InvalidOrderItemException {
        for (OrderItem orderItem : orderItems) {
            if (orderItem.getQty() > 100) {
                throw new InvalidOrderItemException(
                    "Order quantity cannot be more than 100, found: "
                    + orderItem.getQty());
            }
            long id = dao.save(orderItem);
            System.out.println("id generated: " + id);
        }
    }
}
```

Упражнение

- Сейчас упражнение – вопрос слушателя
- Если Вы ответите – то смело можете считать, что разобрались в транзакциях.


```
1
2 @Service
3 public class StoreService {
4     @PersistenceContext
5     EntityManager entityManager;
6
7     @Transactional(propagation = Propagation.REQUIRED)
8     public void createPerson() throws Exception {
9         Personal personal = new Personal();
10        personal.setName("required");
11        entityManager.persist(personal);
12        addNewPerson();
13
14    }
15
16    @Transactional(propagation = Propagation.REQUIRES_NEW, rollbackFor =
Exception.class)
17    public void addNewPerson() throws Exception{
18        Personal personal = new Personal();
19        personal.setName("name_required_new");
20        entityManager.persist(personal);
21        throw new Exception();
22    }
23 }
```

Почему при вызове метода `addNewPerson()` из метода `createPerson()` не создается новая транзакция, несмотря на `Propagation.REQUIRES_NEW`.

И даже при наличии `rollbackFor = Exception.class` в базу коммитится две сущности `Personal`

TransactionTemplate

```
1 @Override
2 public final void updateGreet(final Greeter g, final String newTarget) {
3     transactionTemplate.execute(new TransactionCallbackWithoutResult() {
4         @Override
5         protected void doInTransactionWithoutResult(TransactionStatus status) {
6             Greeter greet = em.merge(g);
7             greet.setTarget(newTarget);
8
9             status.setRollbackOnly();
10        }
11    });
12 }
```

```
1 @Override
2 public final List<Greeter> getGreetings() {
3     TransactionTemplate tx = new TransactionTemplate(transactionManager);
4     tx.setReadOnly(true);
5     return tx.execute(status -> em.createQuery("from Greeter", Greeter.class)
6         .getResultList());
7 }
```



Ваши вопросы?

Если что – их можно задать потом

БЫТ-practices

- Помните всегда на каком уровне работает Ваша база
- Когда пишете запрос – помните про вложенность и уровни изоляции
- Всегда про это вспоминайте когда изменяете запрос
- Дефекты труднообнаружимы. Тестами не покрывается(
- Ну и если хотите пройти собеседование, то это просто как минимум нужно знать)



Ваши вопросы?

Если что – их можно задать потом

Домашнее задание

- Сегодня нет, догоняем хвосты



Опрос !!!

Пожалуйста, пройдите опрос.

**Спасибо за внимание
В меру безопасности вам**

